VHDX parser and format

How Artemis opens a VHDX and turns it into a guest disk. The parser lives under `forensics/src/accessor/disk/vhdx` and follows Microsoft MS-VHDX. libyal's libvhdi notes are the references cited in the source comments. This describes the `accessor-vhdx` branch.

> **Version 1 scope.** Fixed and dynamic disks are read. Differencing disks are rejected. A non-empty log is reported as `pending_log` and is not replayed. A damaged BAT entry fails the open. The virtual disk may be up to 64 TiB.

Module map

`vhdx/mod.rs` declares the modules. `format.rs` turns `DiskFormat::Vhdx` into a `DiskReader::Vhdx`. `image.rs` is the path wrapper. `reader.rs` is the only type that implements `Read` and `Seek` for the guest.

File	Role	Entry points
identifier.rs	File type identifier at offset 0	`parse_header`
header.rs	The two 4 KiB image headers	`parse_image_header`, `active_head`, `vhdx_crc32c`
region.rs	The two 64 KiB region tables	`parse_region_table`, `active_regions`, `parse_entry`
metadata.rs	Virtual disk parameters	`parse_metadata`, `chunk_ratio`, `validate_flags`
bat.rs	Block allocation table and guest mapping	`bat_table_bytes`, `parse_bat`, `VhdxBat::map_offset`
reader.rs	Open, log check, guest Read and Seek	`VhdxReader::open`, `read_mapping`, `log_range`
image.rs	Absolute path and DiskImage	`VhdxDisk::new`, `open_reader`, `image`

First megabyte of the file

MS-VHDX 2.2 fixes the layout of the first 1 MiB. Everything the parser needs to find the BAT and the metadata is inside it. `VhdxReader::open` refuses a file shorter than 320 KiB, which is the end of the second region table.

Offset	Size	Object	Read by
0	64 KiB	File identifier	`parse_header`, 520 bytes
64 KiB	64 KiB	Header copy 1	`active_head`, first 4 KiB
128 KiB	64 KiB	Header copy 2	`active_head`, first 4 KiB
192 KiB	64 KiB	Region table copy 1	`active_regions`
256 KiB	64 KiB	Region table copy 2	`active_regions`
320 KiB	rest of 1 MiB	Reserved	not read

Source: MS-VHDX 2.2. The header and region copies are the updates described in 2.3 and 2.4. Payload blocks, the log, the BAT, and the metadata region all start at or after 1 MiB.

Open sequence

`VhdxReader::open` in `reader.rs` is the whole parse. Each later step uses only what the previous step returned. A failure here is a volume error and the image does not open.

Step	Function	What it decides
1	`image_size`	File length. Seek to the end, then back to 0.
2	`parse_header`	Signature `vhdxfile` and the UTF-16 creator.
3	`active_head`	The header copy with the higher sequence number.
4	`active_regions`	BAT and metadata region locations. Copies must match.
5	`region_in_file`	Both regions fit in the file.
6	`log_range`	Log is 1 MiB aligned and does not overlap either region.
7	`parse_metadata`	Block size, virtual size, sector sizes, disk id, chunk ratio.
8	`bat_table_bytes` then `parse_bat`	How many BAT bytes to read, then the block map.

File identifier

MS-VHDX 2.2.1 and libyal section 2. The first 8 bytes are the little-endian signature `0x656c696678646876`, the ASCII bytes `vhdxfile`. `parse_header` in `identifier.rs` checks that with `nom_u64`, then takes the next 512 bytes as a UTF-16LE creator string. `extract_utf16_string` turns that into `VhdxReader::creator`. The rest of the 64 KiB identifier area is reserved and is not parsed.

Image header

MS-VHDX 2.3.1. Two copies, at 64 KiB and 128 KiB. `parse_image_header` requires 4096 bytes, signature `0x64616568` (“head”), format version 1, and log version 0. `vhdx_crc32c` zeros bytes 4 through 7 and computes the Castagnoli CRC the spec stores in the checksum field. A mismatched checksum rejects that copy.

`active_head` keeps the copy with the greater sequence number. One invalid copy is ignored. Both invalid is an error. Equal sequence numbers are an error, which matches the spec and rejects a header pair that was copied mid-update. The fields kept on `VhdxHeader` are `sequence`, `file_write`, `data_write`, `log`, `log_version`, `version`, `log_length`, and `log_offset`. `image_metadata` copies the three GUIDs out as strings.

`has_pending_log` is true when the log GUID is not nil. MS-VHDX 2.3.1 says a zero log GUID means the log is empty and must not be replayed. `active_head` emits a tracing warning and still returns the header. Nothing in `open` replays log entries. The flag is published as `VhdxImageMetadata.pending_log`.

Region table

MS-VHDX 2.4. Copies at 192 KiB and 256 KiB, each 64 KiB. `parse_region_table` checks signature `0x69676572` (“regi”), the same CRC as the header, a reserved field of zero, and an entry count of at most 2047. Each entry is 32 bytes: GUID, file offset, length, flags. `parse_entry` requires a 1 MiB aligned offset at or past 1 MiB, a non-zero 1 MiB aligned length, no overflow, and no duplicate GUID. The required bit is flags bit 0. Any other flag bit is an error. `check_overlaps` rejects intersecting ranges.

`active_regions` requires the two known regions and returns them on `VhdxRegionTable`. An unknown region is accepted when its required bit is clear and rejected when the bit is set. If both copies parse and differ, the file is rejected. One bad copy is ignored.

Region	GUID	Constant use
BAT	`2DC27766-F623-4200-9D64-115E9BFD4A08`	`active_regions`, `bat_guid`
Metadata	`8B7CA206-4790-4B9A-B8FE-575F050F886E`	`active_regions`, `meta_guid`

Source: MS-VHDX 2.4. The GUIDs are constructed with `Uuid::from_u128` in `region.rs`. libyal section 4 describes the same table.

Metadata region

MS-VHDX 2.6. `open` reads `min(region length, 1 MiB)` from the metadata file offset and passes that buffer to `parse_metadata`. The table itself occupies the first 64 KiB. Item values start at offset 65536 or later, which is why a 64 KiB read is not enough.

The signature is `0x617461646174656d` (“metadata”). The reserved 16-bit field must be zero. At most 2047 entries. Each entry is a GUID, offset, length, and flags. `parse_entry` rejects a zero length, an offset inside the 64 KiB table, a range past the buffer, a duplicate GUID, a non-zero reserved field, and flag bits outside `IsUser`, `IsVirtualDisk`, and `IsRequired`.

Required items

`required_item` fails the open if any of these five GUIDs is missing. `is_required_virtual_item` requires `IsVirtualDisk` and `IsRequired` on each of them. File parameters are the exception: flags equal to `IsRequired` alone (`0x4`) are accepted, which is what QEMU writes. `IsUser` and `IsVirtualDisk` are independent. An entry may set both, one, or neither.

Item	GUID	Parser	Value
File parameters	`CAA16737-FA36-4D43-B3B6-33F0AA44E76B`	`parse_file_parameters`	Block size and LeaveBlocksAllocated
Virtual disk size	`2FA54224-CD1B-4876-B211-5DBED83BF4B8`	`parse_virtual_disk_size`	Guest size in bytes
Page 83 data	`BECA12AB-B2E6-4523-93EF-C309E000C746`	`parse_disk_id`	16-byte disk identifier
Logical sector size	`8141BF1D-A96F-4709-BA47-F233A8FAAB5F`	`parse_sector_size`	512 or 4096
Physical sector size	`CDA348C7-445D-4471-9CC9-E9885251C556`	`parse_sector_size`	512 or 4096
Parent locator	`A8D35F2D-B30B-454D-ABF7-D3D84834AB0C`	`validate_flags`	Rejected. Differencing is not supported.

Source: MS-VHDX 2.6.2. libyal section 5.2.2 is the comment on `parse_file_parameters`. An unknown item with `IsRequired` set is rejected. An unknown item without that bit is ignored.

Derived values

Block size must be a power of two from 1 MiB through 256 MiB. Virtual size must be non-zero, at most 64 TiB, and a multiple of the logical sector size. `LeaveBlocksAllocated` comes from bit 0 of the file-parameters flags and means the disk is fixed. Bit 1, `HasParent`, is a second differencing check and is rejected.

`chunk_ratio` implements the spec definition. One sector-bitmap block describes 2^{23} logical sectors, so the chunk size is 2^{23} times the logical sector size. The chunk ratio is how many payload blocks fit in that chunk:

```
chunk_ratio = (1 << 23) * logical_sector_size / block_size
```

For 512-byte sectors and 1 MiB blocks the ratio is 4096, and one chunk is 4 GiB of guest data. For 512-byte sectors and 32 MiB blocks the ratio is 128.

Block allocation table

MS-VHDX 2.5 and 2.5.1. libyal section 6. The BAT is an array of 64-bit entries. `split_entry` takes the low 3 bits as the state, the next 17 bits as reserved (must be zero), and the high 44 bits as `FileOffsetMB`. Masking with `!0xfffff` leaves the byte offset, because the field is already in 1 MiB units shifted 20 bits.

The layout is the same for fixed and dynamic disks. A sector-bitmap entry follows every chunk except the last one. The last entry is the last payload block. `bat_entry_count` is:

```
payload_blocks + floor((payload_blocks - 1) / chunk_ratio)
```

`payload_blocks` is `virtual_disk_size` divided by `block_size`, rounded up. `parse_bat` walks that pattern once. There is no separate path for `LeaveBlocksAllocated`. A bitmap entry is read only while payload blocks remain after the chunk.

Payload states

State	Name	parse_payload
0	Not present	`PayloadBlock::Absent`, reads as zeros
1	Undefined	Absent. The file offset is ignored.
2	Zero	Absent, reads as zeros
3	Unmapped	Absent, reads as zeros
4, 5	Reserved	Error
6	Fully present	`PayloadBlock::Present` after `validate_present`
7	Partially present	Error. Differencing only.

Source: MS-VHDX 2.5.1, payload BAT entry states. States 1 and 3 may carry a stale `FileOffsetMB`. Hyper-V does this after TRIM. The parser does not return those bytes.

Sector bitmap states

`parse_bitmap` runs on the entries between chunks. State 0 is not present and its offset is ignored. State 6 is present: the parser checks a 1 MiB range with `validate_present` and then discards the bitmap, because a non-differencing disk has no parent to consult. The spec marks state 6 invalid for fixed and dynamic files. Any other state is an error.

How much of the BAT is read

`bat_table_bytes` returns the entry count times 8. It rejects a table larger than 67,125,247 entries, which is 537,001,976 bytes. That is a 64 TiB disk with 1 MiB blocks and 512-byte sectors: 67,108,864 payload entries plus 16,383 bitmap entries. `open` reads `min(region length, that size)` from the BAT file offset. `parse_bat` then requires the buffer to be at least that long.

`validate_present` checks each present payload or bitmap block. The file offset must be at least 1 MiB, the range must fit in the file, and it must not overlap the metadata region, the BAT region, or the log. Those occupied ranges come from `occupied_ranges`. After the walk, `check_present_overlaps` sorts the present ranges by file offset and compares neighbors. Two sorted ranges overlap only when an adjacent pair overlaps.

`VhdxBat` stores one `PayloadBlock` per payload block, in guest order. `map_offset` divides the guest offset by the block size, clips the returned length at the end of the block and at the end of the virtual disk, and returns `VhdxMapping::Zero` or `VhdxMapping::Data`.

Log

MS-VHDX 2.3.2 and 2.3.3 describe the log and require replay before guest I/O when the log GUID is not zero. This parser does not implement replay. `log_range` still checks the geometry: a zero `log_length` means there is no region. Otherwise the offset and length must be 1 MiB aligned, the offset must be at least 1 MiB, the range must fit in the file, and `ranges_overlap` must say it misses both the BAT and the metadata region. The log is then added to the occupied list so a payload block cannot point into it.

Guest bytes for blocks written while the log was active can differ from what the host last flushed. `pending_log` on the image metadata is the signal for that. The on-disk BAT and metadata are what get parsed.

Guest reads

`VhdxReader` implements `Read` and `Seek`. The seek position is a guest offset, not a file offset. `seek_guest` handles `Start`, `Current`, and `End` against `virtual_size` through `checked_seek`. Seeking does not move the underlying file.

`read_guest` stops at the end of the virtual disk. It fills the caller's buffer by calling `read_mapping` once per block boundary. `read_mapping` asks `map_offset` for the current guest position. A zero mapping fills the buffer. A data mapping seeks the file to `file_offset` plus the offset within the block, then `read_exact`. `file_position` remembers where that read finished. The next present read skips the seek when the file cursor is already there. A failed seek or read clears `file_position`. A zero mapping does not, because it does not move the file.

Where the guest disk goes

`VhdxDisk::new` requires an absolute host path. `open_reader` opens the file and calls `VhdxReader::open`. `image` builds `DiskImageMetadata` with format, `file_size`, `virtual_size`, and `DiskImageDetails::Vhdx(VhdxImageMetadata)`.

`DiskFormat::open_reader` in `format.rs` boxes that reader as `DiskReader::Vhdx`. `logical_sector_size`, `read`, and `seek` on `DiskReader` forward to it. `DiskSource::inspect` opens the disk, takes that sector size, and runs `inspect_disk` and `identify_disk`. NTFS and ext4 are then opened through `PartitionReader` at the partition's byte offset. The scheme name is `vhdx`, parsed in `accessor/location/scheme.rs`.

References

- [MS-VHDX] Virtual Hard Disk v2 (VHDX) File Format, Microsoft Open Specifications. Sections used above: 2.2 file layout and identifier, 2.3 headers and log, 2.3.3 log replay, 2.4 region table, 2.5 BAT, 2.5.1 entry states, 2.6 metadata. Document set: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-vhdx/. BAT: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-vhdx/af7334e6-ad2c-4378-9b81-afc1334a6ee7. Bitmap states: https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-vhdx/61251993-f2da-47af-b6ab-e8a8bd299094.
- libvhdi, Virtual Hard Disk version 2 (VHDX) image format: https://github.com/libyal/libvhdi/blob/main/documentation/Virtual%20Hard%20Disk%20version%202%20(VHDX)%20image%20format.asciidoc. The source comments point at section 3 (image header), section 4 (region table), section 5 and 5.2.2 (metadata and file parameters), and section 6 and 6.1 (BAT and BAT entry).